

TME 10 : Processus Gaussien

Crazy Gaussians

The goal of this section is to understand how the covariance matrix of a Gaussian influences the distribution of dimensions. We will first study the 2D case, then the N -dimensional case.

You will need the functions : `multivariate_normal` from `scipy.stats`, which allows computing, among other things, the density of a Gaussian, and `multivariate_normal` from `numpy.random`, which allows drawing random points from a Gaussian more easily than the previous function. If you later receive a message indicating that your covariance matrix is singular (i.e., non-invertible), adding a small random noise to the diagonal will make it invertible.

First, consider three Gaussians in 2D such that :

- The first one has independent axes (diagonal covariance matrix with $\sigma = 1$ on the diagonal).
- The second one has its two dimensions weakly correlated (for example, $cov(x_1, x_2) = 0.1$).
- The third one has its two dimensions highly correlated (for example, $cov(x_1, x_2) = 0.9$).

For each of them :

- Represent the density distribution in 2D space (using `np.meshgrid` and `plt.contour`).
- Plot on the same figure a thousand points sampled according to the distribution.
- Plot some points of the distribution according to the following representation : for a point $\mathbf{x} = (x_1, x_2)$ of the Gaussian, the plot corresponds to the segment $[(1, x_1), (2, x_2)]$.

Observe the differences between each plot according to the covariance matrix values. How does a low covariance manifest ? A high one ?

For a few dimensions D (10, 100, 200), generate some covariance matrices such that all covariances have the same value except for the diagonal, which is fixed at 1. Represent a few samples according to the last 1D representation : for a sampled point $\mathbf{x} = (x_1, \dots, x_D)$, plot the curve $[(1, x_1), (2, x_2), \dots, (D, x_D)]$ (which we will call the *1D view* of the sample). What do you observe depending on the covariance value ?

Covariance Function (Kernel)

Implement the following three kernels :

- The linear covariance : $k_{lin}(\mathbf{x}^1, \mathbf{x}^2) = \sigma^2 \langle \mathbf{x}^1, \mathbf{x}^2 \rangle$
- The exponential kernel : $k_{rbf}(\mathbf{x}^1, \mathbf{x}^2) = \sigma^2 e^{-\frac{1}{2l^2} \|\mathbf{x}^2 - \mathbf{x}^1\|^2}$
- The periodic kernel : $k_{per}(\mathbf{x}^1, \mathbf{x}^2) = \sigma^2 e^{-\frac{2}{l^2} \sin^2(\pi \|\mathbf{x}^2 - \mathbf{x}^1\|/p)}$

Your functions must handle data matrices at all times (of size $N \times D$ —number of samples by number of dimensions). You can use the function `pairwise_distances` from `sklearn.metrics`.

Plot the covariance matrix for a discretization of a fixed 1D domain (for example, $[-5, 5]$) using `plt.imshow`. Observe the effect of parameterization and kernel choice. Also, plot the 1D view of some samples for each covariance matrix.

Gaussian Process and Regression

We assume we have a dataset $D = \{\mathbf{x}^i, y^i\}_{i=1}^N$, where each $y^i = f(\mathbf{x}) + \epsilon$, with f unknown and $\epsilon \sim \mathcal{N}(0, \sigma^2)$. Denoting $\mathbf{x}$ as the training points, $\mathbf{y}$ as the training labels, $\mathbf{x}^t$ as the test points, and $\mathbf{y}^t$ as the labels to infer, Gaussian process regression establishes through marginalization that $p(\mathbf{y}^t | \mathbf{x}, \mathbf{y}, \mathbf{x}^t) \sim \mathcal{N}(\boldsymbol{\mu}_t, K_t)$ with :

$$\boldsymbol{\mu}_t = K_*^t [K + \sigma^2 I]^{-1} \mathbf{y}, \quad K_t = K_{**} - K_*^t (K + \sigma^2 I)^{-1} K_*$$

where K is the covariance matrix between training points, K_* is the covariance matrix between training and test points, and K_{**} is the covariance matrix between test points (assuming centered data). The covariance matrix of $\begin{bmatrix} \mathbf{x} \\ \mathbf{x}^t \end{bmatrix}$ is written as :

$$\begin{bmatrix} K & K_* \\ K_*^t & K_{**} \end{bmatrix}.$$

Write a function that takes as input the data, test points, kernel, and noise estimate σ , and returns the estimated values of $\mathbf{y}^t$ and the associated covariance. Test your function on one of the previous examples by providing only a few training points. On a graph, plot the 1D view of the sample, the training points, the estimation $\boldsymbol{\mu}^t$ from your function, and a confidence interval of $\pm 2 * \sqrt{\text{var}(\mathbf{x})}$ (using `plt.fill_between`). Vary the number of training points and adjust the hyperparameters of the kernels. Try regressing the function $x \sin(x)$. Study the effect of hyperparameters and kernel choice. Try on a real dataset (e.g., Boston Housing).